



Trabalho prático n.º 1 – Interface de linha de comando

Aprendizagens

- Executar comandos e navegar na árvore de ficheiros usando interface de linha comando
- Utilizar as ajudas em ambiente de linha de comando
- Compilar e correr programas em código C através da linha de comando

Introdução

Nas aulas práticas de programação vai trabalhar-se em Linux, através da linha de comando (CLI), usando a shell **Bash**. Trabalhar em linha de comando é uma competência fundamental para qualquer programador, sobretudo em servidores e ambientes de desenvolvimento onde nem sempre existe interface gráfica.

 Para uma introdução aos comandos da Bash, consulte: [Introdução à Bash — Parte 1](#) e [Parte 2](#).

Abrir um terminal:

Sistema

Linux

Windows

MacOS

Sem acesso a nenhum dos anteriores

Como abrir

Ctrl-Alt-T

Instalar o WSL e abrir uma janela WSL

⌘-Espaço → escrever “Terminal”

shell.cloud.google.com (requer conta Google)

Cada linha de comando a executar é indicada com um \$ no início (não escrever o \$, apenas o que vem a seguir):

```
$ pwd
```

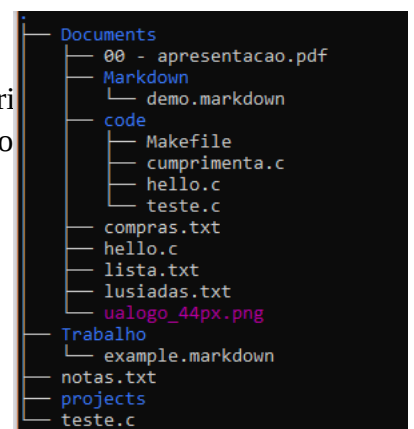
Atividades

Preparar os ficheiros de trabalho

```
$ cd
$ wget -O tudo.zip https://tinyurl.com/41988Programacao
$ unzip tudo.zip
$ tree
```

Isto descarrega e descompacta o conjunto de ficheiros e pastas necessários já existir na sua conta. O comando `tree` mostra toda a árvore de ficheiro

1 Documentação sobre `wget`: `wget --help` ou `man wget`.





💡 Sugestão: use a tecla TAB para autocompletar nomes de comandos e ficheiros — experimente escrever `unz` e pressionar TAB.

Explorar comandos com as ajudas da Bash

Antes de continuar, descubra por si mesmo o que fazem os comandos `pwd`, `man` e `wget`, **sem os executar na sua função principal**. Todo o comando Unix aceita a opção `--help`; existe também o comando `man <comando>` para documentação mais detalhada.

Comando	O que descobriu que faz	Como descobriu (comando usado)
<code>pwd</code>		
<code>wget</code>		

Listar e explorar ficheiros

Na raiz da sua área de trabalho, descubra o comando adequado para cada tarefa:

Tarefa	Comando usado	Resultado
Listar todos os ficheiros na raiz		
Listar apenas os ficheiros <code>.txt</code>		
Listar apenas os ficheiros <code>.c</code>		
Imprimir o conteúdo de <code>notas.txt</code> no ecrã		

Renomear pastas

Para que alguém que fale inglês perceba a estrutura de ficheiros, altere o nome da pasta Documentos para Documents.

Tarefa	Comando
Renomear Documentos → Documents	
Confirmar que a alteração foi feita	

Explorar a pasta Documents

`$ cd Documents`

Tarefa	Comando usado	Resultado / resposta
Listar os ficheiros contidos nesta pasta		
Verificar o tipo de cada ficheiro (comando <code>file</code>)		

Altere a extensão de um ou dois ficheiros para algo incorreto (por exemplo, `.pdf` → `.png`) e volte a verificar o tipo com `file`. O que observa?

R.: Apenas o nome altera-se, pois ao ver o conteúdo do ficheiro este continua igual.

Nesta pasta existem dois ficheiros chamados `hello.c`, em subpastas diferentes.

Tarefa	Comando	Resultado
Localizar os dois ficheiros <code>hello.c</code>	<code>find . -iname *.c</code>	

**Tarefa**

Ver as diferenças entre eles
(diff)

Comando**Resultado****Navegar na árvore de pastas****Tarefa**

Entrar na pasta code
Recuar um diretório
Confirmar em que pasta se está agora
Voltar à pasta raiz da sua conta

Comando

Na árvore de ficheiros existe um conjunto de ficheiros com extensão markdown. Encontre-os todos.

Comando: `find . -name "*.markdown"` Resultado: `./Trabalho/example.markdown./Documents/Markdown/demo.markdown`

Compilação

Ao contrário do Python, o código C tem de ser traduzido para linguagem máquina antes de correr — processo chamado **compilação**, feito pelo compilador gcc. Sempre que o código-fonte (.c) for alterado, é preciso compilar de novo antes de executar a nova versão.

Para executar um programa compilado, é preciso escrever ./nome_do_programa (e não apenas nome_do_programa), porque a Bash só procura programas nas pastas listadas na variável PATH, e não na pasta atual, por razões de segurança.



Pode ver o conteúdo da variável PATH com: `echo $PATH`

Compilar e correr um programa

Mude para a pasta code dentro de Documents.

```
$ gcc -o teste teste.c
$ ./teste
```

Pergunta

Resultado da execução de ./teste
Que instrução usa o C para imprimir no ecrã?

Resposta

Bem vindos a programação!
`printf()`

Repita o processo (compilar e correr) para o programa cumprimenta.c. Verifique o seu conteúdo.

Pergunta

Resultado da execução de ./cumprimenta
Que instrução usa o C para ler do teclado?

Resposta

Olá, Fontinha! A viagem em programação começa aqui!
`scanf("%49s", nome);`

Editar sem compilar: o que acontece?

Esta atividade demonstra, na prática, a relação entre o código-fonte (.c) e o programa compilado.

1. Com teste já compilado (passo anterior), corra-o novamente e confirme o comportamento atual.

Comando: `printf("Bem vindos a Programação!");`

Resultado: `Bem vindos a programação!`



2. Abra `teste.c` e altere algo simples no seu comportamento (por exemplo, o texto da mensagem impressa no ecrã). Grave o ficheiro. **Não compile.**

O que alterou no código? printf("Bem vindos a UA");

3. Corra novamente `./teste`, sem compilar.

Comando: printf("Bem vindos a UA");

Resultado: Bem vindos a programação!

O resultado reflete a alteração feita? Porquê?

R.: _____

4. Agora compile o programa (`gcc -o teste teste.c`) e corra-o de novo.

Comando: _____

Resultado: _____

O resultado reflete agora a alteração? O que mudou em relação ao passo anterior?

R.: _____

Reflexão

Antes de terminar, reveja os comandos que utilizou durante o trabalho.

Identifique:

- um comando que já conhecia;
- um comando que aprendeu a utilizar;
- um comando que descobriu através da documentação;
- uma situação em que teve de experimentar diferentes possibilidades até encontrar uma solução.

Não é necessário apresentar uma explicação extensa. O objetivo é identificar as estratégias utilizadas para resolver os problemas propostos.



Atividade extra — um programa que gera programas

Dentro da pasta code, crie uma pasta OCC e mude para lá:

```
$ mkdir OCC
$ cd OCC
$ wget https://ioccc.org/2012/endoh2/endoh2.c
$ gcc -o endoh2 endoh2.c
$ ./endoh2
```

O resultado da execução é, ele próprio, código C. Guarde-o num novo ficheiro, compile-o e corra-o:

```
$ ./endoh2 > p1.c
$ gcc -o p1 p1.c
$ ./p1
```

Repita o processo mais uma vez, a partir do resultado de p1:

```
$ ./p1 > p2.c
$ gcc -o p2 p2.c
$ ./p2
```

Pode continuar-se *ad infinitum* (ou quase...). O que está a observar?

R.: _____